

FATMA AKKOCA

101406349

Assignment 2 - Code Explanation

1. How have we used classes for our project to reuse code?

In this project, we used Object-Oriented Programming (OOP) to avoid repeating code and to make our game structure more organized. Instead of writing similar code for Hero and Monster separately, we created a Character class that has shared properties like `combat_strength` and `health_points`. Both Hero and Monster inherit from Character, so we didn't have to define these attributes twice. This way, if we ever need to change how characters work, we can do it in one place instead of updating multiple files. This makes the code cleaner, reusable, and easier to maintain.

2. Provide one line of code, as an example, where code is shared between the monster class and the hero class.

```
super().__init__()
```

3. What is the benefit of using complex getters and setters?

Using complex getters and setters helps keep our attributes safe by preventing direct access. For example, we use `@property` to create a getter for `health_points`, so it can only be accessed through a controlled method. With setters, we can add validation to make sure values are correct, like preventing negative health points. This makes the code more secure, avoids unexpected errors, and keeps everything organized.

4. If we didn't use try-except blocks, what would be the problem?

If we didn't use try-except blocks, our program could crash whenever an unexpected error happens. For example, if someone tries to set `health_points` to a negative number or a string instead of an integer, the program would throw an error and stop working. Using try-except allows us to catch these errors, display a meaningful message, and keep the game running smoothly without unexpected crashes.

5. How could we use the name of the operating system or the version of Python in our game to prevent errors?

We can use the OS name and Python version to make sure our game runs in a compatible environment. For example, some Python features might not work in older versions, so we can check the Python version before running the game and warn the user if an update is needed. Similarly, if our game requires OS-specific file paths or commands, we can detect the operating system and adjust the code accordingly. This helps prevent compatibility issues and ensures the game works smoothly on different systems

6. What's another piece of information we could save inside `save.txt`?

In addition to the total number of monsters killed, we could save other important game data in `save.txt`. For example, we could store the hero's remaining health points so that if the game is restarted, the hero doesn't always start from full health. Another idea is to save the total number of games played, which can help track progress. This way, we can make the game more dynamic and allow players to continue from where they left off

7. New Feature Proposal

a. Think of 1 new feature you can add to the game that could use list comprehension and nested loops. For now just write 1 sentence that describes the feature:

Now add your new feature description here: Feature Name: `Monster Waves`

Instead of fighting one monster at a time, the hero will face waves of monsters. Each wave consists of a group of monsters, and after defeating one wave, a new, stronger wave appears. The hero must survive multiple waves to win the game.

a) Using a list comprehension loop:

We create multiple waves of monsters using a list comprehension:

```
waves = [[Monster() for _ in range(3)] for _ in range(2)] # 2 waves, each with 3 monsters
```

This allows us to have structured waves of enemies where each group is stronger than the last.

b) Using nested conditional statements:

Each wave fights the hero until either all monsters are defeated or the hero is defeated. If a wave is cleared, the next wave begins.

```
for wave in waves:
```

```
    print("New wave of monsters has appeared!")
```

```
for monster in wave:
    while monster.health_points > 0 and hero.health_points > 0:
        hero.hero_attacks(monster)

        if monster.health_points > 0:
            monster.monster_attacks(hero)

    if hero.health_points == 0:
        print("Hero has fallen! Game Over!")
        break

    if hero.health_points == 0:
        break

print("All waves defeated! Hero wins!")
```
